



SCReAM Congestion Control

Our implementation vs UMN's reimplementation

A controlled, like-for-like comparison — same video, same network, fair config

What we compared

Both stacks stream the **same 60 s camera clip** on one machine over loopback.

The network is squeezed the **same way for both**: a Linux shaper replays three recorded 5G patterns —

- HO — a sudden handover drop
- RB — bumpy tower-sharing
- CQI — smooth signal drift (high capacity)

Same bitrate limits. UMN's delay signal turned on so the comparison is fair.

HOLDING CONSTANT

Video

1280×1024, 10 fps, 60 s

Host

single machine, loopback

Shaper

tc on lo, 5G traces ×1/3

Reps

3 per trace

Varies

only the SCSReAM build

Result: ours uses the network better

63%

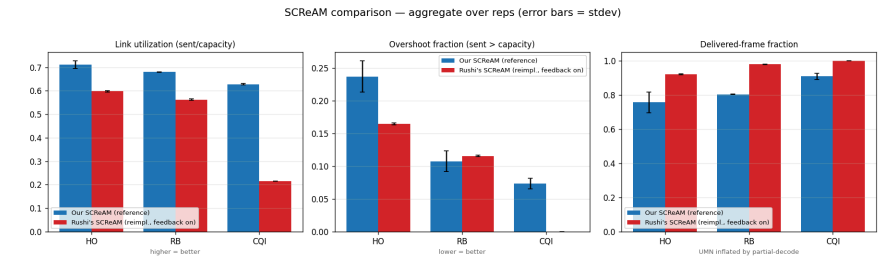
OURS — bandwidth used
on the high-capacity link

22%

UMN — bandwidth used
same link, same moment

Ours raises and lowers its sending rate to match the link.

UMN's sends a near-constant **~780 kbps on every trace** — even when the capacities differ 2.5×. It doesn't follow the network.



Why: the controller can't drive the encoder

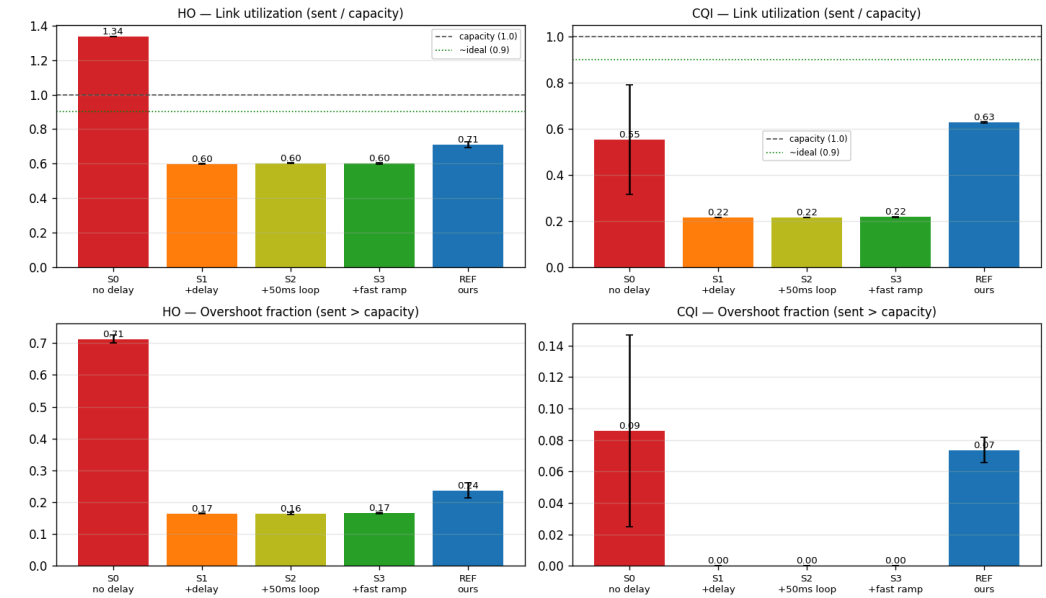
UMN's controller decides a rate correctly — but that decision never reaches the video encoder.

The encode loop writes the new rate onto an already-running software encoder, which the library ignores; the rate is fixed when the encoder opens.

We ruled out timing: making the loop 4× faster changed nothing (S2, S3 in the chart).

Analogy a thermostat that correctly says "turn down the heat" — but the wire to the furnace is cut.

Factor isolation ladder — adding back one capability per rung
(Rushi's SReAM walked toward our reference)



Two independent reasons ours is more mature

● 1 · Measured

Open control loop

On the software path we tested, the controller's rate is computed but never applied — the loop is open. Ours is closed: the framework applies the rate to the encoder automatically.

Evidence: sent rate exceeded the commanded rate in ~89% of decisions.

● 2 · From code review

Simpler control law

UMN's is a fixed-gain multiply-up ($\times 1.05$) / down ($\times 0.8$) loop on yes/no signals, with no memory. The reference uses a congestion window and a continuous queue-delay trend with adaptive steps.

Not measured here — the wiring gap masked it. From reading the source.

Takeaway

Our SReAM tracks a changing link; UMN's holds a near-constant rate.

The cause is implementation maturity — a closed control loop and the reference algorithm — not a flaw in SReAM itself.

HONEST CAVEATS

- UMN's deployed NVENC encoder uses the same no-reopen code path we tested, so the gap very likely applies there too — only the unused VA-API path reopens. Not yet run on NVENC hardware.
- We measured network behavior (how well it uses the link), not delivered video quality (PSNR).
- Single-host loopback reproduces bandwidth and delay, not real 5G radio effects.

H5 — Our SCReAM adapts to changing bandwidth better than UMN's reimplementation — because UMN's controller can't drive its encoder, not because of the algorithm

supported Project-internal cross-implementation comparison against UMN Teleop-Gopher-streamer (scream-integration branch 502533c)

Claim

When the available bandwidth keeps changing, our SCReAM raises and lowers its sending rate to match what the link can carry, so it uses most of the available bandwidth. UMN's reimplementation instead holds a roughly constant sending rate, so it leaves most of a fast link unused. The reason is what we call a controller-to-encoder gap: the congestion controller (the part that decides how fast to send) picks a sending rate, but that decision is never applied to the video encoder, so the encoder keeps producing video at roughly the same rate no matter what. This is a plumbing problem inside UMN's implementation — not a flaw in the SCReAM algorithm itself, and not a matter of how often its control loop runs. (A Glossary at the end defines any remaining term.)

Prediction

Both stacks stream the same 60 s clip on one machine over loopback (sender and receiver are two processes on the same host). The network is squeezed the same way for both: a Linux traffic shaper on the loopback interface replays recorded 5G bandwidth patterns — a sudden handover drop (HO), bumpy tower-sharing (RB), and smooth signal drift (CQI) — each scaled to 1/3 capacity so the link is tight. Both get the same bitrate limits, and UMN's network_time_sync flag is on so its delay signal (the cue SCReAM uses to sense a building queue, and back off) actually works. We expect: our SCReAM's sending rate rises and falls with the available bandwidth, so it uses most of what the link offers — high "utilization", the share of available bandwidth actually used — while UMN's stays near a constant rate across the three traces, whose capacities differ about 2.5x. We also expect the gap to remain when UMN's control loop is sped up (re-checking every 50 ms instead of 200 ms) and made to ramp up faster — which would rule out loop timing as the cause — and to line up with the controller-to-encoder gap measured directly: the rate actually sent exceeds the rate the controller asked for, in most decisions.

Verdict rule

Supported when (a) on the high-capacity CQI trace our utilization is at least 2x UMN's; (b) UMN's mean sent rate barely changes across the three traces (spread $\leq 15\%$), showing it does not track capacity; (c) speeding up UMN's loop to 50 ms and raising its ramp gain do not raise its CQI utilization (timing ruled out); and (d) the diagnostic shows the actual sent rate exceeding the controller's commanded target by more than 1.3x in most decisions. Refuted if the utilization gap closes once the delay signal is on, or if faster timing recovers utilization. Untested if the comparison metrics are missing.

Main claim

MECHANISM

With both implementations configured fairly, our SCReAM changes its sending rate to match the available bandwidth, while UMN's holds a near-constant rate. The cause is a controller-to-encoder gap: UMN's controller decides on a rate, but the encode loop tries to apply it by writing `codec.bit_rate` on an already-running software encoder — which libav ignores, since a software encoder's bitrate is fixed when it opens. So the rate is set once at startup and never updated. This is a wiring problem in the implementation, not a flaw in the SCReAM algorithm or a matter of how often the control loop runs.

Supporting evidence

- Link utilization (share of available bandwidth used) on the high-capacity CQI trace: ours ~63%, UMN ~22%.
- UMN's sending rate stays near 776–780 kbps on all three traces, whose capacities differ about 2.5x — it does not rise when more bandwidth is available.
- Speeding up UMN's control loop (200 ms to 50 ms) and ramping faster did not raise its utilization — loop timing is not the cause.
- Measured directly: the actual sent rate exceeded the controller's commanded rate by more than 1.3x in about 89% of decisions (the controller-to-encoder gap); UMN's packet-loss congestion signal also never fired.
- Pinpointed in code: the encode loop (`encoder.py _encode_stream_pyav`) sets `codec.bit_rate` on a live encoder; only a full encoder reopen actually changes the rate, and `libvpx` never triggers that reopen. UMN's separate controller-thread design hand-wires this connection per codec; our SCReAM is a built-in pipeline element whose rate is applied by the framework.

Findings and Limitations

Findings

- Utilization (the share of available bandwidth actually used) on the high-capacity CQI trace: ours ~63%, UMN ~22%.
- UMN's SCReAM sends a near-constant ~776–782 kbps across the HO, RB, and CQI traces, whose capacities differ about 2.5x; it does not raise its rate when more bandwidth is available.
- The shortfall is a controller-to-encoder gap (the controller's chosen rate is not applied to the encoder), not the SCReAM algorithm or its loop timing.
- Pinpointed in code: in `encoder.py` the encode loop applies the controller's rate by writing `codec.bit_rate` on an already-running software encoder, which libav ignores (a software encoder's bitrate is fixed when it opens). Only a full encoder reopen changes the rate, and the `libvpx` path never triggers that reopen — so the rate is set once at startup. This is a wiring problem in the implementation, not the SCReAM algorithm.
- Architecture difference: UMN runs the controller as a separate thread that leaves a per-frame "budget" the encode loop must apply by hand, per codec; our SCReAM is a built-in pipeline element whose rate the media framework applies to the encoder automatically. The reference + framework-integrated design avoids this class of bug.
- Algorithm fidelity (from source-code review, NOT measured in this experiment — the wiring gap masked it): the reference SCReAM is a queue-delay-gradient controller with a congestion window, bytes-in-flight limiting, and adaptive step sizes; UMN's is a fixed-gain multiply-up (x1.05) / multiply-down (x0.8) loop on binary signals with no memory. So the maturity case has two independent legs — a simpler control law (code-level) and an open control loop on the software path (measured). See the algorithm-comparison table.

- An earlier version of this comparison showed UMN overshooting the link badly; that was our configuration mistake (network_time_sync was off, which disables UMN's delay signal). With it on, UMN no longer overshoots — confirming that result was an artifact, not a property of UMN's controller.
- UMN's delay congestion signal works once network_time_sync is on; its packet-loss signal never fires in these runs.

Limitations

- Single-host loopback reproduces the bandwidth and delay faithfully but not real 5G radio effects (interference, scheduling, retransmission at the radio layer).
- We measured network behavior (utilization, overshoot, sent rate), not delivered picture quality (PSNR/SSIM). "Uses the link better" is a statement about bandwidth use, not yet a measured video-quality difference.
- UMN's controller is an early reimplementaion (its own code marks receiver feedback as a future phase). This is a snapshot of branch 502533c, not a statement about the SCReAM algorithm in general.
- Encode path (corrected): only UMN's VAAPI path reopens the encoder when the bitrate changes. NVENC — which UMN actually deploys — takes the SAME PyAV path as the libvpx encoder we tested: it applies the controller's rate with a live codec.bit_rate write and never reopens (NVENC uses in-band keyframes). libav fixes an encoder's bitrate when it opens and ignores later writes, so the controller-to-encoder gap almost certainly affects UMN's deployed NVENC path too — not just the software path. (An earlier draft wrongly implied the hardware path escapes the gap; that described VAAPI, which UMN does not deploy.)
- Open question / verify next: NVENC behavior here is inferred from the code (PyAV path + libav's open-time bitrate), not measured — there is no NVIDIA GPU on the bench. Confirming it needs an NVENC run on real hardware under the same traces.
- Open question / verify next: we measured network behavior (utilization), not delivered video quality. The honest quality metric is PSNR/SSIM of the received video vs the source, on both stacks.

Figures

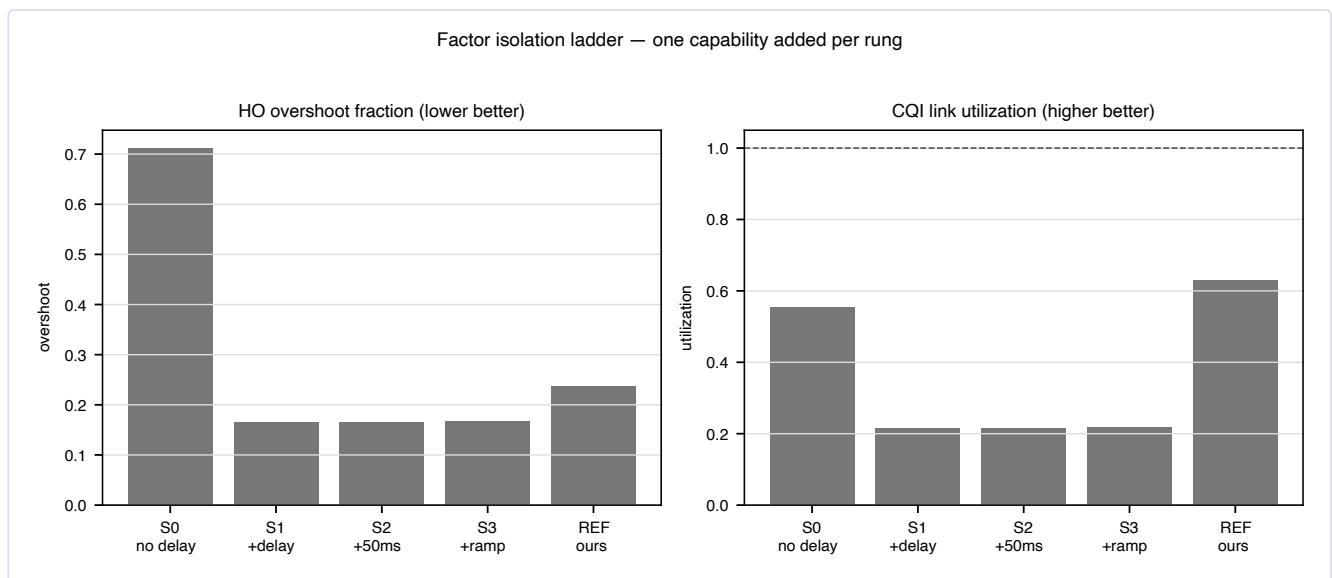


Figure H5-1. Isolating the cause. Each bar adds one capability to UMN's SCReAM: S0 (none) → S1 (+delay signal) → S2 (+faster 50 ms loop) → S3 (+faster ramp); REF is our reference SCReAM. Left: overshoot (the share of time spent sending faster than the link can carry — lower is better) drops once the delay signal is on at S1, and the faster-timing steps S2/S3 do not change it. Right: utilization (the share of available bandwidth used — higher is better) on the high-capacity CQI trace stays low through S1/S2/S3, so faster timing does not help; only the reference implementation reaches high utilization.

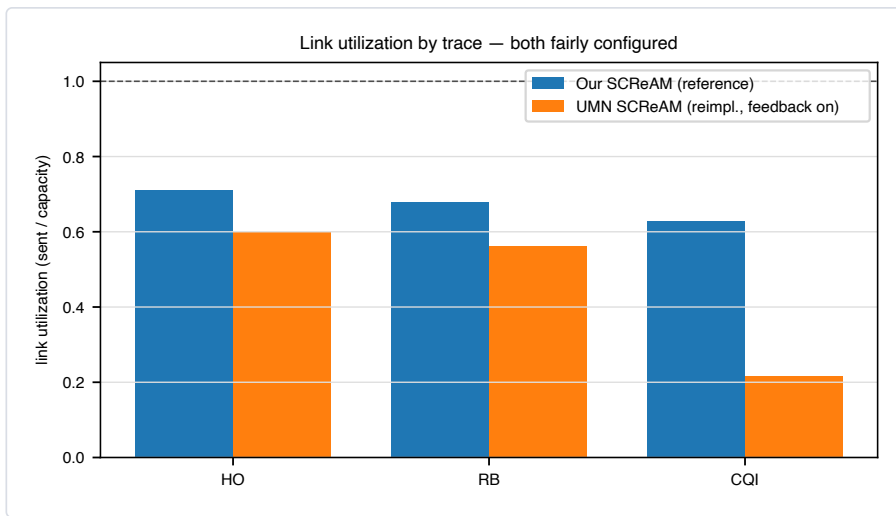


Figure H5-2. Share of available bandwidth used (utilization), per trace, with both implementations configured fairly. Ours rises with the available bandwidth (most visibly on the high-capacity CQI trace); UMN's stays low because its sending rate is held near a constant regardless of the link. The dashed line marks 1.0 — using all the available bandwidth.

Tables

h5_fair_comparison

trace	capacity_kbps	ours_utilization	umn_utilization	ours_sent_kbps	umn_sent_kbps	ours_overshoot	umn_overshoot
HO	2,387	0.711	0.598	1,020	776	0.237	0.165
RB	1,825	0.680	0.563	1,040	776	0.108	0.116
CQI	4,710	0.629	0.216	2,339	780	0.074	0.000

h5_isolation_ladder

rung	ho_overshoot	cqi_utilization	cqi_sent_kbps
S0 no delay	0.712	0.553	2,109
S1 +delay	0.165	0.216	780
S2 +50ms	0.164	0.216	780
S3 +ramp	0.166	0.216	782
REF ours	0.237	0.629	2,339

h5_implementation_maturity

dimension	umn_reimplementation	ours_reference
Algorithm	Hand-written Python, SCRAM-inspired; its own code calls it a first pass for hardware integration	Ericsson reference SCRAM (C++), via the gstscream plugin
Does the rate decision reach the encoder?	No on the software path — assigns codec.bit_rate on a running libvpx encoder, which libav ignores	Yes — applied live through GStreamer's encoder property; the framework reconfigures the encoder
Congestion signals used	Delay only (and only with network_time_sync on); loss signal never fires; receiver feedback marked a future phase	Delay and loss together, via standard RTCP feedback
Packet pacing	None — packets leave as the encoder emits them	SCRAM paces its own RTP send queue

dimension	umn_reimplementation	ours_reference
Integration	Separate controller thread leaving a per-frame budget the encode loop must apply correctly per codec	Built-in pipeline element; the media framework guarantees the rate reaches the encoder

h5_algorithm_comparison_code_review

aspect	umn_reimplementation	ours_reference
Control variable	One scalar target rate	A congestion window + bytes-in-flight limit, then a rate derived from it
Delay handling	Binary: is one-way delay over the 120 ms threshold?	A queue-delay trend/gradient vs a target, scaled continuously
Rate increase	Blind x1.05 every 200 ms tick, regardless of history	Adaptive: fast when far below the last good point, gentle near it
Rate decrease	Fixed x0.8 (x0.7 on loss)	Proportional to how far queue delay exceeds the target
State kept	Just the current target rate	cwnd, smoothed RTT, queue-delay stats, reference rate, in-flight bytes
Extras	None	ECN/L4S marking, RTP-queue-aware, fast-start

Experimental setup

Required metrics

- utilization
- overshoot_frac
- mean_sent_kbps

Glossary

Congestion controller

The part of a video streamer that decides how fast to send so it does not overflow the network. SCRaM and GCC are two such controllers.

SCRaM

A delay-based congestion controller. "Our SCRaM" is the reference implementation from Ericsson (via the gstscream plugin); "UMN's SCRaM" is a separate Python reimplementation in the Teleop-Gopher-streamer.

Link utilization

The share of the available bandwidth the sender actually used (sent rate divided by link capacity). About 1.0 is ideal; well below 1.0 wastes bandwidth; above 1.0 means sending more than the link can carry.

Overshoot

The fraction of the run spent sending faster than the link can carry. Sending above capacity makes packets pile up and drop, so lower is better.

Controller-to-encoder gap

When the congestion controller computes a target sending rate but that target is not actually applied to the video encoder, so the encoder keeps producing video at a different (here, roughly constant) rate. The controller decides correctly; the encoder does not follow.

Open vs closed control loop

A control loop is "closed" when the controller's decision actually changes the thing it controls (here, the encoder's output rate). It is "open" when the decision is computed but never applied, so it has no effect. UMN's software encode path is effectively open; our framework-integrated path is closed.

Software vs hardware encoder

A software encoder (e.g. libvpx for VP8) runs on the CPU; a hardware encoder (e.g. NVENC) runs on a dedicated GPU block. They reconfigure differently — UMN's bitrate-change works on its hardware path (which reopens the encoder) but not on the software path tested here.

Delay signal (one-way delay)

SCReAM's main congestion cue — how long a packet takes to travel from sender to receiver. Rising delay means a queue is building, so the sender should slow down.

Loss signal

The fraction of packets that did not arrive — a second congestion cue SCReAM can use alongside delay.

network_time_sync

A UMN config flag that stamps a send timestamp on each packet so the receiver can compute one-way delay. With it off, UMN's SCReAM has no delay signal and runs effectively blind.

Trace (HO / RB / CQI)

A recording of how a real 5G link's bandwidth changed over time, replayed to drive the bottleneck. HO = handover between cell towers (sharp drops); RB = resource-block sharing with other users (bumpy); CQI = channel-quality drift (smooth, high average).

x0.33

The trace's bandwidth scaled to one-third of the original, so the link is tight enough for this clip to actually stress the controller.

tc on loopback (tc-on-lo)

Linux traffic-control shaping applied to the loopback network interface, used to impose the trace's changing bandwidth and a fixed delay on traffic between the two local processes.

Isolation ladder (S0-S3, REF)

A controlled experiment that starts from UMN's weakest setup and adds one capability per step - S0 (no delay signal) to S1 (+delay signal) to S2 (+faster 50 ms loop) to S3 (+faster ramp), with REF being our reference SCReAM. The change at each step shows how much that one factor contributes.

Reproducibility

This page is generated from `specs/hypotheses/h5.yaml` + `analysis/hypotheses/results/h5_report.json` + figures under `analysis/hypotheses/results/`.

```
python3 analysis/hypotheses/build_reports.py
python3 analysis/hypotheses/build_pages.py
python3 analysis/hypotheses/h5_umn_scream_implementation_gap.py
```

Source: Project-internal cross-implementation comparison against UMN Teleop-Gopher-streamer (scream-integration branch 502533c)

Generated by `analysis/hypotheses/build_pages.py`. Do not hand-edit; modify `specs/hypotheses/h5.yaml` or the template instead.